

CreatorPulse - Sustainable Influencer Acquisition Analytics

> A data-driven analytics product that helps social-commerce marketing teams understand which influencer engagement patterns are associated with sustainable customer acquisition.

1. Product Overview

Problem Statement

A social commerce platform captures influencer campaign metrics, referral traffic, and purchase behaviour, but marketing teams still cannot determine which creator engagement patterns produce sustainable customer acquisition.

Product

Build an interactive data analytics product called:

CreatorPulse

CreatorPulse analyzes influencer campaign performance, referral traffic, and customer purchase behaviour to help marketing teams understand: - Which creators generate meaningful conversions - Which engagement patterns correlate with purchases - Which creators generate repeat customers - Which campaigns generate high-value customers - Whether high engagement actually translates into sustainable acquisition - Where users drop out of the acquisition funnel - How creator performance changes over time The application should transform raw social-commerce data into: **Data -> Cleaning -> Validation -> Analysis -> KPIs -> Visualizations -> Business Insights**

2. IMPORTANT DEVELOPMENT PHILOSOPHY

This is a Kalvium Sprint 1 DATA PRODUCT. Do NOT build an unnecessarily complicated full-stack application. The primary goal is: 1. Data ingestion 2. Data profiling 3. Data cleaning 4. Data validation 5. Data transformation 6. EDA 7. Business metric calculation 8. SQL analysis where appropriate 9. Visualization 10. Interactive Streamlit dashboard 11. Business insights The application should demonstrate that the team understands the DATA behind the problem. The application is NOT primarily: - An influencer social media platform - A CRM - An influencer marketplace - A generic admin dashboard - A chatbot - An AI wrapper - A CRUD application AI may be added later as an optional enhancement, but the analytical pipeline must work without an LLM.

3. CORE BUSINESS QUESTION

The entire product must answer:

"Which influencer engagement patterns are associated with sustainable customer acquisition, measured through conversion, repeat purchasing, retention, and customer value?" The primary stakeholder is: **Marketing / Growth Team** They need to make decisions about: - Creator selection - Campaign allocation - Content strategy - Referral strategy - Customer acquisition - Retention - Campaign effectiveness

4. SECONDARY BUSINESS QUESTIONS

The application should answer these questions wherever the available dataset supports them.

Q1 - Creator performance

Which creators generate the most: - Referral traffic? - Purchases? - Revenue? - New customers? - Repeat customers?

Q2 - Engagement vs conversion

Does higher engagement correspond to higher conversion? Analyze relationships between: - Likes - Comments - Shares - Saves - Engagement rate - Referral clicks - Purchases - Conversion rate

Q3 - Engagement vs sustainability

Does high engagement correspond to sustainable customer acquisition? Compare: - Engagement rate - First purchase conversion - Repeat purchase rate - Customer retention - Customer lifetime value This is one of the most important analytical questions.

Q4 - Creator quality

Are there creators who generate: - Extremely high engagement - High clicks - High first-time purchases but relatively poor repeat purchase behaviour? These creators should be identified as a business pattern, NOT automatically labelled as "bad." The application should show the underlying metrics and let the marketing team interpret them.

Q5 - Content type

If the dataset contains content type: Compare: - Review - Tutorial - Unboxing - Lifestyle - Discount - Product demonstration - Giveaway - Other against: - Engagement - Click-through rate - Conversion - Repeat purchase - Revenue

Q6 - Campaign performance

Which campaigns generate: - Highest conversion - Highest revenue - Lowest CAC - Highest repeat purchase rate - Highest revenue per customer

Q7 - Funnel performance

Where do customers drop off? Example: Impressions -> Engagement -> Referral Click -> Product Visit -> Purchase -> Repeat Purchase

Q8 - Time trends

How do creator/campaign metrics change over time? Analyze: - Engagement - Clicks - Purchases - Revenue - Repeat purchase rate - Retention

5. IMPORTANT DATA PRINCIPLE

NEVER fabricate analytical conclusions. If the dataset does not contain a variable required for a

metric, do NOT invent it. For example: If customer-level purchase history is unavailable, do not pretend that the application can calculate true repeat purchase rate. Instead show:

"Repeat purchase analysis unavailable because customer-level purchase history is not present in the current dataset." The application must clearly distinguish:

Observed

Directly calculated from the dataset.

Derived

Calculated from available columns.

Estimated

Only when a clearly documented methodology exists.

Unavailable

Cannot be calculated from the available data.

6. EXPECTED DATA MODEL

The actual dataset may differ. FIRST inspect the available files and infer the schema. Do not blindly assume these columns exist. The product ideally works with some combination of:

Creator / Influencer Data

``text creator_id creator_name followers creator_category creator_tier

Campaign Data

campaign_id campaign_name creator_id campaign_start_date campaign_end_date content_type
platform campaign_budget

Engagement Data

impressions likes comments shares saves engagements

Referral Data

referral_clicks landing_page_visits referral_code

Purchase Data

customer_id order_id creator_id campaign_id purchase_date order_value product_id

1. IF DATA IS NOT PROVIDED

If no real dataset is present in the repository:
Create a clearly labelled DEMO/SYNTHETIC dataset for development.

Do NOT present synthetic results as real business findings.

The UI should display:

"Demo Mode - Insights are generated from synthetic development data."

Create enough data to demonstrate:

- Multiple creators
- Multiple campaigns
- Multiple customers
- Multiple purchases
- Different engagement patterns
- Different conversion rates
- Repeat customers
- Different campaign types
- Dates across several months

Prefer at least:

20+ creators 30+ campaigns 5,000+ customers 10,000+ purchases

if practical.

The synthetic data should contain realistic relationships rather than purely random numbers.

1. DATA PIPELINE

Implement a clear data pipeline.

Raw Data -> File Loading -> Schema Detection -> Dataset Profiling -> Data Cleaning -> Data Type Standardisation -> Missing Value Handling -> Duplicate Detection -> Validation -> Multi-source Merge -> Feature Engineering -> Analytics Dataset -> Dashboard

Keep raw data untouched.

Recommended structure:

data/ raw/ processed/ demo/

1. DATA PROFILING

Create a profiling module that checks:

- Number of rows
- Number of columns
- Data types
- Missing values
- Duplicate rows
- Unique values
- Numeric ranges
- Date ranges
- Potential outliers

Example:

Dataset Quality Rows 125,430 Columns 18 Missing cells 2.7% Duplicate rows 43 Date range Jan-Jun 2026 Quality status: Yes Schema valid Yes Date column valid ! 3.1% missing referral clicks Yes No duplicate order IDs

1. DATA DICTIONARY

Create:

docs/data_dictionary.md

For every field explain:

Field Meaning Type Business Role creator_id Unique creator string Dimension campaign_id Campaign identifier string Dimension impressions Number of impressions integer Metric likes Likes generated integer Metric comments Comments integer Metric shares Shares integer Metric referral_clicks Referral traffic integer Metric customer_id Customer identifier string Entity order_value Purchase value float Revenue

Do not invent meanings.

Infer from the dataset and document assumptions.

1. DATA CLEANING

Implement robust cleaning.

Handle:

Missing values

For every important column determine:

- Why is it missing?
- Can it be safely filled?
- Should the row be excluded?
- Should the metric be marked unavailable?

Do NOT blindly replace every missing value with zero.

For example:

Missing comments does not necessarily mean:

comments = 0

unless the dataset definition supports that assumption.

1. DUPLICATE DETECTION

Check duplicates at different levels.

Examples:

Duplicate rows Duplicate order_id Duplicate customer_id + order_id Duplicate creator_id + campaign_id + date

Document the chosen deduplication strategy.

1. DATA TYPES

Standardise:

- Dates -> datetime
- Counts -> integer

- Revenue -> float
- IDs -> string
- Percentages -> numeric
- Categories -> normalized strings

Handle:

Rs.10,000 10,000 10000

appropriately if such formatting exists.

1. FEATURE ENGINEERING

Create meaningful derived metrics.

Only create metrics when their denominator exists and is valid.

Engagement Rate

Potential definition:

$(\text{likes} + \text{comments} + \text{shares} + \text{saves}) / \text{impressions} \times 100$

Document the exact formula.

If the dataset already provides engagement rate, compare/validate it instead of blindly replacing it.

Click Through Rate

$\text{referral_clicks} / \text{impressions} \times 100$

Conversion Rate

Depending on the available funnel:

$\text{purchases} / \text{referral_clicks} \times 100$

or:

$\text{purchases} / \text{landing_page_visits} \times 100$

The denominator must be explicitly documented.

Revenue per Customer

$\text{total revenue} / \text{unique customers}$

Average Order Value

$\text{total revenue} / \text{number of orders}$

Repeat Purchase Rate

If customer-level order history exists:

$\text{customers with 2+ purchases} / \text{customers with at least 1 purchase} \times 100$

Customer Retention

If dates support it:

Calculate a clearly documented retention definition.

Example:

30-day repeat:

customers who make another purchase within 30 days of first purchase / customers who made a first purchase

Do not claim retention if the dataset does not support the required time window.

Customer Lifetime Value

Only calculate a basic observed value if sufficient purchase history exists.

For example:

total revenue generated by customer

or an explicitly documented approximation.

Do not pretend this is a predictive LTV model.

1. FUNNEL ANALYSIS

Create an acquisition funnel.

Preferred structure:

Impressions -> Engagement -> Referral Clicks -> Product Visits -> Purchases -> Repeat Purchases

For each stage show:

- Volume
- Percentage of previous stage
- Drop-off percentage

Example:

Impressions 500,000 -> 8.4% Engagement 42,000 -> 35.7% Referral Clicks 15,000 -> 28.0%
Purchases 4,200 -> 34.5% Repeat Customers 1,449

Only display stages that actually exist in the data.

1. EDA

Implement exploratory analysis.

Distribution analysis

Analyze:

- Engagement rate
- Referral clicks

- Conversion rate
- Revenue
- Order value
- Repeat purchase rate

Use appropriate plots.

1. CORRELATION ANALYSIS

Create a correlation matrix for relevant numerical variables.

Example:

Conversion

Engagement 0.42 Shares 0.58 Comments 0.31 Clicks 0.71 Followers 0.09

IMPORTANT:

Correlation does NOT imply causation.

The UI should include a small note:

"Correlation indicates association, not causation."

1. BEHAVIOURAL SEGMENTATION

If customer-level data exists, segment customers into categories such as:

One-time Buyer Repeat Buyer High-value Buyer

Possible segmentation:

One-time

1 purchase

Repeat

2+ purchases

High-value

Top 20% by observed customer revenue

Document the exact thresholds.

Do not use arbitrary labels without documentation.

1. CREATOR SEGMENTATION

Creators can be grouped using meaningful categories.

Example:

Micro Mid-tier Macro

ONLY if follower counts exist and thresholds are documented.

Alternatively use data-driven quartiles:

Q1 Q2 Q3 Q4

This avoids arbitrary follower thresholds.

1. CREATOR ANALYSIS

Create a creator-level analytical table.

Example:

Creator	Engagement	CTR	Conversion	Revenue	Repeat Rate
Creator A	8.2%	4.1%	6.8%	Rs.2.4L	31%
Creator B	5.4%	6.2%	8.1%	Rs.3.1L	42%

Allow sorting by any metric.

1. CREATOR PATTERN ANALYSIS

Analyze patterns such as:

High engagement / low conversion

Potential pattern:

High Engagement + Low Conversion

Low/moderate engagement / high repeat purchase

Potential pattern:

Moderate Engagement + High Repeat Purchase

High conversion / low retention

Potential pattern:

High First Purchase + Low Repeat Purchase

These should be presented as observed segments/patterns, not as causal explanations.

1. BUSINESS KPI DEFINITIONS

Create a dedicated KPI documentation section.

At minimum:

Total Revenue

Sum of valid order values.

Total Orders

Count of valid orders.

Unique Customers

Count distinct customer IDs.

New Customers

Customers making their first observed purchase in the dataset.

Repeat Customers

Customers with 2+ observed purchases.

Repeat Purchase Rate

Repeat Customers / Customers

Engagement Rate

Document formula based on available engagement fields.

CTR

Referral Clicks / Impressions

Conversion Rate

Document denominator.

CAC

If campaign spend exists:

Campaign Spend / New Customers

If campaign spend does NOT exist:

Do not invent CAC.

Display:

CAC unavailable - campaign spend data not provided.

1. SUSTAINABLE ACQUISITION

Do NOT create a meaningless arbitrary "AI score."

Instead create a transparent analytical framework.

Sustainable acquisition should be examined through multiple dimensions:

Acquisition + Conversion + Repeat Purchase + Customer Value + Retention

Display these dimensions separately.

Example:

Creator X Engagement Rate 8.7% Conversion Rate 5.9% Repeat Purchase 38.4% Revenue / Customer Rs.1,920 30-Day Repeat 29.1%

Then generate an insight:

"Creator X shows moderate-to-high engagement and a relatively high repeat-purchase rate in the observed data."

Do NOT say:

"Creator X is the best creator."

Do NOT create rankings unless explicitly required by the product requirements.

1. MAIN STREAMLIT APP

Use Streamlit.

The application should be professional and presentation-ready.

Streamlit supports multipage applications and current documentation recommends the `st.Page + st.navigation` approach for customizable navigation.

Use that architecture where appropriate.

Reference: <https://docs.streamlit.io/develop/concepts/multipage-apps/page-and-navigation>

1. APPLICATION STRUCTURE

Recommended:

creatorpulse/

app.py

pages/ overview.py creator_analysis.py campaign_analysis.py funnel_analysis.py
customer_analysis.py data_quality.py insights.py

src/ **init**.py data_loader.py cleaning.py validation.py features.py metrics.py analytics.py insights.py
utils.py

data/ raw/ processed/ demo/

sql/ creator_metrics.sql campaign_metrics.sql funnel_metrics.sql customer_metrics.sql

docs/ data_dictionary.md metric_definitions.md assumptions.md analysis_notes.md

tests/ test_cleaning.py test_metrics.py test_validation.py

requirements.txt README.md .gitignore

1. PAGE 1 - EXECUTIVE OVERVIEW

This is the landing page.

Header:

CreatorPulse

Sustainable Influencer Acquisition Analytics

Subtitle:

Understand which creator engagement patterns are associated with customers who convert, return, and generate value.

KPI Cards

Display:

Total Revenue Unique Customers Total Orders New Customers Repeat Purchase Rate Average
Order Value

Only display metrics supported by the dataset.

Date Filter

Allow:

All Time Last 7 Days Last 30 Days Last 90 Days Custom Range

If the dataset is historical rather than live, label appropriately.

Global Filters

Sidebar:

Date Range Creator Campaign Platform Content Type Creator Segment

All dashboard visualizations should respond to these filters.

1. OVERVIEW CHARTS

Include:

Revenue over time

Line chart.

Purchases over time

Line/bar chart.

Repeat purchase trend

Line chart if data supports it.

Acquisition funnel

Funnel visualization.

Engagement vs conversion

Scatter plot.

Engagement vs repeat purchase

Scatter plot.

1. PAGE 2 - CREATOR ANALYSIS

Title:

Creator Performance

Display creator-level metrics.

Filters:

Creator Creator Segment Content Type Campaign Date

Main table:

Creator Detail

When a creator is selected:

Display:

Creator Overview

Then:

Engagement trend

Referral traffic trend

Revenue trend

Conversion trend

Repeat purchase trend

Content performance

1. PAGE 3 - CAMPAIGN ANALYSIS

Display:

Campaign Creator Content Type Spend Impressions Engagement Clicks Purchases Revenue New
Customers Repeat Purchase Rate

Charts:

- Campaign revenue
 - Campaign conversion
 - Campaign engagement
 - Campaign repeat purchase
 - Campaign funnel
-

1. PAGE 4 - CUSTOMER BEHAVIOUR

If customer-level data exists.

Display:

Unique Customers One-time Buyers Repeat Buyers High-value Customers Average Orders /
Customer Average Revenue / Customer

Charts:

Purchase frequency distribution

1 purchase 2 purchases 3 purchases 4+ purchases

Customer revenue distribution

Repeat purchase trend

Cohort/retention analysis if sufficient dates exist

1. PAGE 5 - FUNNEL ANALYSIS

Show:

Impressions -> Engagement -> Referral Clicks -> Product Visits -> Purchases -> Repeat Purchases

For each stage:

Users Conversion % Drop-off %

Allow filtering by:

- Creator
- Campaign
- Content type
- Date

This should allow the marketing team to identify where acquisition loses users.

1. PAGE 6 - DATA QUALITY

This page demonstrates the data engineering work.

Show:

Dataset rows Dataset columns Missing values Duplicates Invalid records Outliers Date range

Create a data quality table:

Check Status Details Missing values ! 2.3% Duplicate orders Yes 0 Invalid dates Yes 0 Negative revenue ! 3 Missing creator IDs Yes 0

This is important for the Kalvium Data Product evaluation.

1. PAGE 7 - INSIGHTS

This page should answer the actual business problem.

Do not simply show charts.

Generate concise evidence-based observations.

Example:

INSIGHT 01 Engagement does not increase at the same rate as conversion. Creators in the top engagement quartile generated 8.1% average engagement but only 5.2% conversion. Creators in the second engagement quartile generated 6.7% engagement and 6.4% conversion. This indicates that engagement volume alone may not explain conversion performance in the observed dataset.

1. INSIGHT GENERATION

Build a deterministic insight engine first.

Example rules:

if engagement_high and conversion_low: insight = "High engagement with relatively low conversion was observed." if conversion_high and repeat_rate_low: insight = "High first-purchase conversion but relatively low repeat purchasing was observed." if repeat_rate_high and engagement_moderate:

insight = "Some creators show moderate engagement alongside relatively strong repeat purchasing." if clicks_high and purchases_low: insight = "Referral traffic is high relative to purchases, indicating a lower click-to-purchase conversion rate."

Insights MUST contain the underlying metrics.

Never generate unsupported claims.

1. OPTIONAL AI INSIGHTS

Only implement this after the deterministic analytics work is complete.

If an LLM is added:

The LLM should receive structured metrics like:

```
{ "creator": "Creator A", "engagement_rate": 8.2, "conversion_rate": 5.1, "repeat_purchase_rate": 28.4, "revenue": 240000 }
```

The LLM should summarize the provided evidence.

The LLM must NOT invent metrics.

Prompt the model:

"Use only the supplied metrics. Do not infer causality. Do not invent missing information. Clearly distinguish observations from hypotheses."

If no API key is available, the application must continue working normally.

1. VISUAL DESIGN

The app should look like a professional analytics product.

Avoid:

- Excessive emojis
- Huge colorful cards
- Random gradients everywhere
- Gaming-style UI
- Excessive animations
- Generic AI dashboard aesthetics
- Clutter

Preferred:

Clean Professional Data-focused Minimal Readable Executive-friendly

Use:

- White/light background or restrained dark mode
 - Clear typography
 - Consistent spacing
 - Consistent card styles
 - Subtle borders
 - Professional charts
-

1. COLOR SEMANTICS

Use color meaning consistently.

For example:

Positive:

Green

Warning:

Amber

Problem:

Red

Neutral:

Gray/blue

Do not use random colors for every chart.

1. RESPONSIVE DESIGN

The application should work on:

- Laptop
- Desktop
- Tablet

Prioritize desktop because this is a marketing analytics product.

1. PERFORMANCE

Streamlit reruns the application when users interact with widgets.

Use caching appropriately.

Use:

`@st.cache_data`

for expensive deterministic data-loading and transformation operations.

Do not unnecessarily cache mutable state.

Reference:

<https://docs.streamlit.io/develop/concepts/architecture/caching>

1. CODE QUALITY

The code should NOT be one giant Python file.

Avoid:

`app.py` = 2000 lines

Instead separate:

data loading cleaning metrics analytics visualization UI

Use reusable functions.

Example:

```
def calculate_engagement_rate(df): ... def calculate_conversion_rate(df): ... def  
calculate_repeat_purchase_rate(df): ... def build_creator_metrics(df): ...
```

1. ERROR HANDLING

The app should never crash simply because:

- A column is missing
- A dataset is empty
- A metric cannot be calculated
- A filter returns no rows
- A date is invalid
- An API key is missing

Instead show useful messages.

Example:

No customer-level purchase history is available. Repeat purchase analysis cannot be calculated for this dataset.

1. EMPTY STATES

If a filter returns no data:

Show:

"No records match the selected filters."

Do not show broken charts.

1. METRIC VALIDATION

All calculated metrics should have validation.

Examples:

Conversion rate cannot be > 100% Engagement rate cannot be negative Revenue should not be negative unless refunds are explicitly represented Clicks cannot exceed impressions if the dataset defines impressions as total exposure

Do not automatically delete suspicious values.

Flag them first.

1. SQL REQUIREMENT

Where SQL is useful, create SQL queries demonstrating:

- Filtering
- GROUP BY
- Aggregation
- JOIN
- Window functions
- Ranking

- Business metrics

Example:

```
SELECT creator_id, SUM(order_value) AS revenue, COUNT(DISTINCT customer_id) AS customers FROM purchases GROUP BY creator_id ORDER BY revenue DESC;
```

Another:

```
SELECT creator_id, customer_id, COUNT(*) AS purchase_count FROM purchases GROUP BY creator_id, customer_id;
```

Use SQL for analytical evidence where appropriate.

1. WINDOW FUNCTIONS

Where supported, demonstrate useful analytical SQL.

Example:

```
ROW_NUMBER() RANK() LAG() SUM() OVER(...)
```

Potential use:

Rank creators by revenue Calculate creator revenue trends Calculate customer purchase sequence Calculate first vs subsequent purchases

Do not use window functions merely to show them.

Use them where they solve an actual analytical problem.

1. TESTING

Create tests for important calculations.

Example:

```
test_engagement_rate() test_conversion_rate() test_repeat_purchase_rate()  
test_average_order_value() test_duplicate_detection() test_missing_value_handling()
```

Example:

```
def test_repeat_purchase_rate(): ...
```

Use small controlled datasets.

1. README DOCUMENTATION

The final README must explain:

1. Problem
2. Business question
3. Stakeholder
4. Dataset
5. Data dictionary
6. Data cleaning
7. Feature engineering
8. KPI definitions
9. Analytical methodology
10. Dashboard screenshots

11. Insights
 12. Limitations
 13. Tech stack
 14. Project architecture
 15. How to run
 16. Team contributions
-

1. ASSUMPTIONS DOCUMENT

Create:

docs/assumptions.md

Document every assumption.

Example:

Assumption 1: Conversion rate is defined as purchases / referral clicks. Reason: The dataset does not provide product page visits.

1. LIMITATIONS

The application must clearly communicate limitations.

Potential limitations:

- Observational data does not establish causation
 - Historical data may not represent future campaigns
 - Customer behaviour may be incomplete
 - Attribution may be imperfect
 - Some creators may have insufficient observations
 - Repeat purchase calculations depend on available history
 - CAC requires campaign spend
 - LTV requires sufficient customer history
-

1. DO NOT MAKE THESE CLAIMS

Never say:

"Creator X causes more purchases."

Instead:

"Creator X had a higher observed conversion rate."

Never say:

"High engagement causes customer retention."

Instead:

"Higher engagement was associated with higher/lower repeat purchase rates in the observed data."

Never say:

"This creator is the best."

Instead:

"This creator recorded the highest observed revenue under the selected filters."

1. OPTIONAL CREATOR COMPARISON

Allow users to select 2-4 creators.

Example:

Compare Creators Creator A Creator B Creator C

Comparison:

Metric	A	B	C
Engagement	8.2%	6.1%	5.7%
CTR	4.2%	5.9%	4.7%
Conversion	5.2%	7.1%	6.4%
Repeat Purchase	28%	39%	34%
Revenue	Rs.2.4L	Rs.3.1L	Rs.2.8L

The user should interpret the trade-offs.

1. OPTIONAL QUADRANT ANALYSIS

Create an analytical scatter plot:

X-axis:

Engagement Rate

Y-axis:

Repeat Purchase Rate

Quadrants:

HIGH REPEAT

Moderate engagement / high repeat	High engagement / high repeat

-----+-----> | Low engagement | High engagement / low repeat | / low repeat |
-> LOW REPEAT

This directly connects engagement behaviour to sustainability.

Again, this is descriptive, not causal.

1. OPTIONAL CONTENT ANALYSIS

If content type exists:

Create:

Content Type Performance Content Type Engagement CTR Conversion Revenue Repeat Purchase

Visualization:

Review Tutorial Unboxing Lifestyle Discount

Compare their observed metrics.

1. FILTER ARCHITECTURE

Global filters should be shared.

Recommended:

Date Range Creator Campaign Platform Content Type Customer Segment

Every page should respect global filters where logically applicable.

1. DATA EXPORT

Allow users to download filtered analytical tables.

For example:

Download Creator Metrics CSV Download Campaign Metrics CSV Download Customer Metrics CSV

This makes the product more useful to a real marketing team.

1. PROJECT FLOW

The coding agent should implement in this order.

Phase 1 - Inspect

FIRST:

1. Inspect repository
2. Inspect dataset files
3. Identify schema
4. Identify existing code
5. Identify existing dependencies
6. Identify whether real data exists

Do not start writing the UI before understanding the data.

Phase 2 - Data

Implement:

Loading -> Profiling -> Cleaning -> Validation -> Feature engineering

Phase 3 - Analytics

Implement:

Creator metrics Campaign metrics Customer metrics Funnel metrics Time-series metrics

Phase 4 - Visualisation

Build:

KPIs Charts Tables Funnel Scatter plots Trend analysis

Phase 5 - Streamlit

Build:

Overview Creator Analysis Campaign Analysis Customer Behaviour Funnel Data Quality Insights

Phase 6 - Testing

Test:

Metric calculations Missing values Empty datasets Filters Invalid values

Phase 7 - Polish

Improve:

UI UX Loading states Error states Documentation

1. GIT WORKFLOW

Use small commits.

Examples:

feat: add dataset profiling feat: implement data cleaning pipeline feat: add creator metrics feat: add campaign analytics feat: add funnel analysis feat: build overview dashboard feat: add creator analysis page feat: add data quality page feat: add insight engine test: add metric validation docs: add metric definitions style: improve dashboard layout

Do not make one massive commit containing the entire application.

1. KALVIUM CONCEPT MAPPING

The product should demonstrate relevant Sprint 1 concepts.

Data Ingestion

Concepts:

4-5

Data Cleaning

Concepts:

6-16

Especially:

- Dataset profiling
- Data dictionary
- Missing values
- Data types
- Duplicates
- Text normalization
- Date transformations
- Outliers
- Validation
- Multi-source merging
- Feature engineering

Analysis & EDA

Concepts:

17-26

Especially:

- Distribution analysis
- Correlation
- GroupBy
- Time series
- Behavioural analysis
- Funnel analysis
- KPI design
- Root cause analysis

SQL

Concepts:

27-34

Especially:

- Business metrics
- Filtering
- Aggregation
- Joins
- Window functions
- Insight validation

Visualization

Concepts:

35-40

Especially:

- KPI cards
- Interactive plots
- Business storytelling
- Executive reporting

Streamlit

Concepts:

41-46

Especially:

- App structure
 - Filters
 - Interactive widgets
 - Session state where necessary
 - Real-time KPI dashboard
 - Monitoring
-

1. IMPORTANT: BUILD FOR DEMONSTRATION

Every major feature should be easy to explain in a 2-5 minute video.

For example:

Feature:

Creator Analysis

Explain:

"We first clean and validate the creator-level data. Then we group campaign and purchase information by creator. We calculate engagement rate, CTR, conversion, revenue and repeat purchase rate. The dashboard lets the marketing team compare creators and identify engagement patterns associated with sustainable acquisition."

This should be visible in the code.

1. FINAL DEMO FLOW

The final presentation should follow this story:

Step 1

Introduce the problem.

Marketing teams have lots of influencer data but cannot easily identify which engagement patterns produce sustainable customers.

Step 2

Show the data.

We combine creator engagement, referral traffic and purchase behaviour.

Step 3

Show data quality.

Before analysis we profile, clean and validate the data.

Step 4

Show KPIs.

The dashboard summarizes acquisition, conversion and customer value.

Step 5

Show funnel.

We identify where users drop between engagement and purchase.

Step 6

Show creator analysis.

We compare engagement against conversion and repeat purchase.

Step 7

Show behavioural analysis.

We distinguish one-time and repeat customers.

Step 8

Show insights.

We identify observed patterns in the data.

Step 9

Explain limitations.

These are observational relationships, not causal conclusions.

1. DEFINITION OF DONE

The project is considered complete only when:

Data

- Raw data is preserved
- Dataset profiling implemented
- Data cleaning implemented
- Missing values handled
- Duplicates checked
- Data types standardized
- Validation implemented
- Feature engineering implemented
- Data dictionary documented

Analytics

- Creator metrics
- Campaign metrics
- Customer metrics
- Funnel metrics
- Time-series metrics
- Engagement analysis
- Conversion analysis
- Repeat purchase analysis where supported
- Correlation analysis
- Behavioural segmentation

SQL

- Aggregation queries
- Joins
- Business metric queries
- At least one meaningful window-function query if supported

Dashboard

- Overview
- Creator analysis
- Campaign analysis
- Customer behaviour
- Funnel analysis
- Data quality
- Insights
- Filters
- CSV export

Engineering

- Modular code
 - Error handling
 - Tests
 - Requirements file
 - Documentation
 - No hardcoded fake business conclusions
 - App runs from a clean environment
-

1. DEVELOPMENT COMMANDS

Create a standard setup.

Example:

```
python -m venv .venv source .venv/bin/activate pip install -r requirements.txt streamlit run app.py
```

If using Windows, document the appropriate activation command as well.

1. REQUIREMENTS

Use a reasonable stack.

Recommended:

Python Pandas NumPy Streamlit Plotly SQLAlchemy / SQLite if SQL storage is required Scikit-learn only if a genuinely useful analytical feature requires it

Avoid unnecessary dependencies.

1. IMPORTANT INSTRUCTION TO THE CODING AGENT

Before changing anything:

1. Inspect the repository.
2. Inspect all available datasets.
3. Inspect existing code.
4. Determine actual columns.
5. Determine actual data relationships.
6. Determine what metrics are genuinely computable.
7. Create a short implementation plan.
8. Then implement incrementally.

Do NOT:

- blindly fabricate schemas
 - fabricate business results
 - invent missing columns
 - add an LLM unnecessarily
 - create fake analytics
 - hardcode dashboard numbers
 - write a 2000-line app.py
 - use random charts simply to make the dashboard look full
 - claim causation from correlation
 - calculate CAC without spend
 - calculate retention without adequate time/customer history
 - calculate repeat purchase without customer-level history
-

1. MOST IMPORTANT PRODUCT PRINCIPLE

The dashboard should not answer:

"Who has the most likes?"

It should answer:

"What observable creator engagement and campaign patterns are associated with customers who convert and continue purchasing?"

The difference between these two questions is the core of the project.

1. FINAL PRODUCT EXPERIENCE

When a marketing manager opens CreatorPulse, they should immediately understand:

"What happened?"

Through KPIs and trends.

"Where did it happen?"

Through creator/campaign filters.

"Why might it be happening?"

Through funnel, correlation and behavioural analysis.

"What pattern should I investigate?"

Through evidence-based insights.

"Can I trust the numbers?"

Through the Data Quality page and metric definitions.

1. FINAL UX

The application should feel like:

A real marketing analytics product

not:

A college assignment with charts.

Prioritize:

Business clarity -> Data correctness -> Analytical depth -> Useful visualizations -> Professional UI

Do not prioritize flashy UI over correct analytics.

One thing I'd do differently while vibe-coding

Don't paste this and tell Claude/Gemini:

"Build everything." That usually produces a huge amount of mediocre code. Instead, give it this README and work **phase by phase**.

Prompt 1 - first interaction

Paste:

Read the README completely. Do not code yet. First inspect the repository, all datasets/files, existing code, and current environment. Tell me the actual data schema, what metrics can genuinely be calculated, what metrics cannot be calculated, and any ambiguities you find. Then propose the implementation plan mapped to the README. Do not invent data columns or business conclusions. Then let it inspect.

Prompt 2 - data layer

Implement Phase 1 and Phase 2 only: data loading, profiling, cleaning, validation, and feature engineering. Do not build the Streamlit UI yet. Run tests and show me the resulting dataset schema and data-quality summary.

Prompt 3 - analytics

Now implement the analytics layer: creator metrics, campaign metrics, customer behaviour, funnel metrics, time-series metrics, correlation analysis and sustainable-acquisition analysis. Every metric must have a documented formula. Test the calculations.

Prompt 4 - dashboard

Now build the Streamlit application according to the README. Make it professional and analytical, not flashy. Connect every chart to the real analytics functions. No hardcoded KPI values.

Prompt 5 - polish

> Act as a skeptical reviewer. Inspect the entire project for incorrect calculations, unsupported claims, data leakage, hardcoded values, broken filters, misleading visualizations, missing error handling and poor code structure. Fix the issues you find.

One particularly important thing for your sprint

Your screenshot shows that **Concept #24 is "KPI Definition & Business Metric Design" and #26 is "Root Cause Investigation Workflow."** Those are actually very well suited to your problem. So don't make the project merely:

CSV -> Pandas -> 10 charts -> Streamlit Make the reasoning visible: ``text RAW DATA
-> DATA QUALITY -> BUSINESS METRICS -> FUNNEL ANALYSIS -> BEHAVIOUR /
SEGMENTATION -> CREATOR PATTERN ANALYSIS -> BUSINESS INSIGHTS

That gives you something you can defend in the PR/video evaluation, rather than just saying "Claude generated this dashboard."

For the Streamlit implementation, the current official docs support both multipage approaches, with `st.Page/st.navigation` being the preferred customizable approach; Streamlit also recommends `st.cache_data` for caching data-returning computations.

[Streamlit multipage apps documentation](#)

[Streamlit caching documentation](#)